Perguntas e respostas — Nodify

Consulta rápida para a apresentação

Respostas curtas para consulta durante a apresentação. Toda informação aqui é verificável no repositório.

Testes

Qual o framework base dos testes?

JUnit 4 (4.13.2) como framework de execução, **MockK** (1.13.13) para substituir dependências e **kotlinx-coroutines-test** para controlar o tempo nos testes assíncronos. Nos testes de interface, **Compose UI Test** e **Espresso**. Todos já estavam no projeto — não adicionamos nenhuma dependência de teste nova.

Por que JUnit 4 e não JUnit 5?

O executor dos testes instrumentados do Android, o `AndroidJUnit4`, é construído sobre o JUnit 4. Adotar o 5 exigiria um plugin Gradle de terceiros e deixaria os testes unitários e os instrumentados em frameworks diferentes.

O que é o MockK e por que precisam dele?

É uma biblioteca que cria objetos falsos no lugar das dependências reais. No teste do `PlayerAtivoViewModel`, por exemplo, o repositório do Spotify é substituído por um falso: assim verificamos que o comando certo foi chamado sem precisar de internet nem de conta real. *Por que MockK e não Mockito:* classes Kotlin são `final` por padrão e o Mockito precisaria de configuração extra; o MockK também simula função `suspend` sem adaptação.

Os testes rodam automaticamente?

Sim. Há integração contínua no GitHub Actions (`.github/workflows/android.yml`): a cada `push` e a cada *pull request*, o servidor roda `ktlintCheck`, `detekt` e `test`. Se qualquer um falhar, o build quebra.

Como vocês testam sem celular?

A camada de regra de negócio não importa nada de Android. O classificador recebe números e devolve um gesto; o ViewModel recebe os repositórios por construtor, substituídos por simulações. Por isso os 186 testes rodam na JVM em menos de 4 segundos.

Qual a cobertura de código?

71,4 % do núcleo lógico e 22 % do aplicativo inteiro, medido com **Kover**. A diferença são cerca de 1.300 linhas de Compose, que descrevem a tela e não têm lógica para testar. Domínio, dados, integração com o Spotify e os 4 ViewModels estão em **100 %**. Comando: `./gradlew koverHtmlReportDebug`

O documento fala em 186 testes, mas você rodou 61. Por quê?

Os 186 são a suíte unitária completa. Os 61 são um recorte das quatro funcionalidades principais, para a demonstração caber no tempo. `./gradlew test` mostra os 186.

Qual a diferença entre os níveis de teste de vocês?

Nível	O que valida	Precisa de aparelho?
Unitário (186)	Uma classe isolada, com dependências substituídas	Não
Integração (33)	Camadas reais conversando: Room, Keystore, SharedPreferences	Sim
Funcional (51)	A tela renderizada, do ponto de vista de quem usa	Sim
Usabilidade	Se a pessoa consegue realizar a tarefa	Sim

Vocês usaram TDD?

Não de forma estrita. Os testes foram escritos sobre a implementação existente, de propósito: o objetivo era registrar os problemas encontrados antes de corrigi-los. Foi assim que surgiram os quatro problemas de usabilidade documentados.

Como sabem que os testes não são superficiais?

Porque vários pegam erros que o compilador não pega. Um exemplo: o teste de unicidade dos rótulos de ação. Se dois rótulos fossem iguais, a conversão devolveria sempre a primeira ação da lista, e o gesto do usuário dispararia o comando errado — sem erro de compilação e sem exceção em execução.

Arquitetura

Qual padrão arquitetural e por quê?

MVVM em quatro camadas (apresentação, processamento, integração e dados). É o padrão recomendado pelo Google e o que o Jetpack Compose assume. A decisão decisiva foi manter a camada de domínio sem nenhum `import` de Android — é isso que permite testar a regra de calibração sem emulador.

Por que injeção de dependência (Hilt)?

Porque é o que permite trocar a implementação real pela simulada no teste. O `PlayerAtivoViewModel` recebe interfaces de repositório pelo construtor; em produção o Hilt injeta a implementação com Retrofit, no teste injetamos uma falsa. Sem isso, a classe criaria a dependência internamente e não haveria como testá-la.

Por que Jetpack Compose e não XML?

Compose é declarativo: a tela é uma função do estado. Combinado com o `StateFlow` dos ViewModels, cada mudança de estado redesenha só o necessário. Também torna as telas testáveis sem inflar layout.

Tecnologia

Por que Kotlin?

É a linguagem oficial do Android desde 2019. Para este projeto pesaram dois pontos: segurança contra nulos verificada em tempo de compilação, e corrotinas, que deixam o processamento de câmera e as chamadas de rede fora da thread principal sem travar a interface.

Como o gesto é reconhecido, tecnicamente?

A CameraX entrega os quadros, o **MediaPipe Face Landmarker** devolve 478 pontos do rosto, e deles estimamos a rotação da cabeça em três eixos — *roll* (inclinar), *pitch* (subir/descer) e *yaw* (virar). Cada ângulo é comparado ao limiar que a calibração mediu para aquele usuário. Se passar, é um gesto.

Por que MediaPipe?

Roda inteiramente no dispositivo, o que é requisito do projeto: nenhuma imagem pode sair do celular. É mantido pelo Google, tem modelo otimizado para tempo real e entrega as 478 marcações necessárias para estimar a pose com precisão suficiente.

Como evitam disparo acidental?

Quatro proteções combinadas: o limiar é individual (75 % da amplitude medida na calibração, com um mínimo de segurança por eixo); o gesto precisa persistir por 2 quadros consecutivos; há intervalo mínimo de 1 segundo entre comandos; e a cabeça precisa voltar para perto da posição neutra antes de um novo disparo ser aceito.

Por que Room e não SQLite direto?

O Room gera o código de acesso a partir de anotações e valida as consultas em tempo de compilação — um erro de SQL aparece ao compilar, não em produção. Também devolve `Flow`, o que faz a tela atualizar sozinha quando o mapeamento muda.

Como integram com o Spotify?

Pela **Spotify Web API**, por REST com Retrofit. A autenticação é **OAuth 2.0 com PKCE**, o fluxo que o Spotify exige para aplicativos móveis — a senha do usuário nunca passa pelo nosso app.

Produto e acessibilidade

Por que o mapeamento gesto→ação é customizável?

Porque é um recurso de acessibilidade, não uma conveniência. Usuários têm capacidades motoras diferentes: quem não consegue virar o rosto para a esquerda precisa associar esse comando a outro gesto. Fixar o mapeamento no código excluiria essas pessoas. A referência é a **WCAG 2.1, critério 2.5.6**.

Como garantem a privacidade?

Nenhum quadro de vídeo ou ponto facial é gravado em disco ou enviado pela rede — todo o processamento é local. A única informação derivada da imagem que sai do pipeline é a luminância média, um número agregado usado para avisar sobre iluminação ruim. Os tokens do Spotify ficam cifrados pelo Android Keystore. É imagem de rosto, dado biométrico sensível pela LGPD.

Qual a latência?

A meta é 500 ms entre o gesto e a ação. Medido no Galaxy S25+, ficou entre **200 e 450 ms**.

Funciona em qualquer celular?

Android 10 (API 29) ou superior, com câmera frontal de pelo menos 720p. A calibração é por aparelho: trocar de celular exige refazer os sete passos, porque a convenção de sinais dos eixos varia entre modelos.

E se a pessoa não conseguir fazer nenhum gesto de cabeça?

O sistema não resolve esse caso — o público-alvo é quem tem mobilidade reduzida mas preserva movimento cefálico. A tela de rastreamento mantém botões manuais como alternativa, e o vocabulário tem nove gestos para que a pessoa use os que conseguir.

Limitações (as perguntas difíceis)

O que não está funcionando?

Quatro itens, todos declarados na seção 5.1 do documento: feedback sonoro por voz, comandos com o Spotify em segundo plano (exigiria um `AccessibilityService`), exclusão dos dados de calibração pela interface, e tela de ajuda. Nenhum foi apresentado como concluído.

Por que exige conta Premium?

É limitação da Spotify Web API, não do aplicativo: os endpoints de controle de reprodução retornam HTTP 403 para contas gratuitas. O app detecta e exibe "Spotify Premium necessário para controle de reprodução" em vez do código de erro.

Por que os 84 testes instrumentados não foram executados?

Faltou aparelho ou emulador disponível no ambiente de desenvolvimento no momento da geração dos relatórios. Eles estão implementados e compilam. As mesmas integrações foram validadas manualmente no Galaxy S25+ — são os casos TI01 a TI11 do documento. Comando para executá-los: `./gradlew connectedDebugAndroidTest`

Qual foi a maior dificuldade técnica?

A convenção de sinais dos eixos de rotação muda de aparelho para aparelho, e a câmera frontal espelha a imagem. Com o sinal fixo no código, o gesto funcionava em um celular e invertia em outro. A solução foi aprender a polaridade de cada eixo durante a calibração, em vez de presumi-la — há teste cobrindo as duas polaridades.